

ITAI 4374: Neuroscience as a Model for AI

Module 04 Assignment Guide

Exploring Spiking Neurons with Wolfram Language

by Hisham D Macaraya

Professor: Patricia McManus

Part A: Solving the LIF Equation with NDSolve

■ Step 1: Verify Your Setup

```
In[2]:=
In[3]:= Print["Wolfram Language ready!"]
Wolfram Language ready!
In[4]:= Print[$Version]
14.3.0 for Linux x86 (64-bit) (July 8, 2025)
```

■ Step 2: Understand the LIF Equation in Wolfram

Wolfram Language excels at solving differential equations directly. Instead of writing a loop like in Python, we can give Wolfram the LIF equation and let NDSolve handle it. The LIF equation as a differential equation:

```
In[5]:= (* The LIF differential equation: *)
(* tau * V'[t] == -(V[t] - Vrest) + R * I[t] *) (* *)
(* tau = membrane time constant (seconds) *)
(* Vrest = resting potential (mV) *) (* R = membrane resistance (MOhm) *)
(* I[t] = input current (nA) *) (* *) (* When V[t] ≥ Vthresh, neuron spikes and resets *)
```

The challenge: NDSolve can't handle the discontinuous reset directly. We will use WhenEvent to detect threshold crossing and reset the voltage.

Tip:

WhenEvent is a powerful Wolfram feature that lets you say: “when this condition becomes true, do this action.” Perfect for spike detection and reset. Step 3: Define Neuron Parameters

The LIF equation models how a neuron’s voltage changes over time. Input current raises the voltage, while the leak term pulls it back toward the resting potential. NDSolve solves the equation numerically, and WhenEvent resets the voltage when it reaches the threshold, allowing the neuron to spike.

■ Step 3: Define Neuron Parameters

Set up your parameters. Create a new cell:

```
In[6]:= (* ===== Neuron Parameters ===== *)
tau = 0.020; (* Membrane time constant: 20 ms *)
Vrest = -70.0; (* Resting potential in mV *)
Vthresh = -55.0; (* Spike threshold in mV *)
Vreset = -75.0; (* Reset potential after spike *)
R = 1.0; (* Membrane resistance, normalized *)

(* ===== Input Current ===== *)
(* Constant current that turns on at t=0.05 and off at t=0.4 *)
Iinput[t_] := If[0.05 ≤ t ≤ 0.4, 18.0, 0.0];

Print["Parameters defined."]
Parameters defined.
```



KNOWLEDGE CHECK

1. The input current is defined as a function of time using If[]. In plain English, what does Iinput[t] return at t = 0.03? At t = 0.2? At t = 0.5?

Iinput[t] returns 0.0 at t = 0.03, because the current has not turned on yet. It returns 18.0 at t = 0.2, because that time is between 0.05 and 0.4. It returns 0.0 at t = 0.5, because the current has already turned off.

2. Why do we use := (delayed assignment) for Iinput instead of = (immediate assignment)?

We use := because Iinput[t] is a time-dependent function and needs to be evaluated each time a value of t is used. Delayed assignment makes Wolfram compute the correct output based on the current time, instead of fixing it too early.

■ Step 4: Solve the LIF Equation

⇒ YOUR TASK 1: Use NDSolve with WhenEvent

Write the NDSolve call to solve the LIF equation. Use WhenEvent to detect when $V[t]$ crosses the threshold and reset it.

```
In[13]:= (* Solve the LIF equation with spike reset *)
spikeCount = 0;
spikeTimes = {};

sol = NDSolve[
{
  (* TODO 1: Write the LIF differential equation *)
  (* tau * V'[t] == -(V[t] - Vrest) + R * Iinput[t] *)
  tau * V'[t] == -(V[t] - Vrest) + R * Iinput[t],

  (* Initial condition: start at resting potential *)
  V[0] == Vrest,

  (* TODO 2: WhenEvent for spike detection and reset *)
  (* When V[t] reaches Vthresh: *)
  (*   - Reset V[t] to Vreset *)
  (*   - Increment spikeCount *)
  (*   - Append t to spikeTimes *)
  (* Hint: WhenEvent[V[t] ≥ Vthresh, *)
  (*   {V[t] → Vreset, *)
  (*   spikeCount++, *)
  (*   AppendTo[spikeTimes, t]} *)
  WhenEvent[
    V[t] ≥ Vthresh,
    {V[t] → Vreset, AppendTo[spikeTimes, t]}
  ]
},
V,
{t, 0, 0.5}
];

spikeCount = Length[spikeTimes];

Print["Neuron fired ", spikeCount, " spikes"];
Print["Spike times: ", spikeTimes];
```

Neuron fired 8 spikes

Spike times:

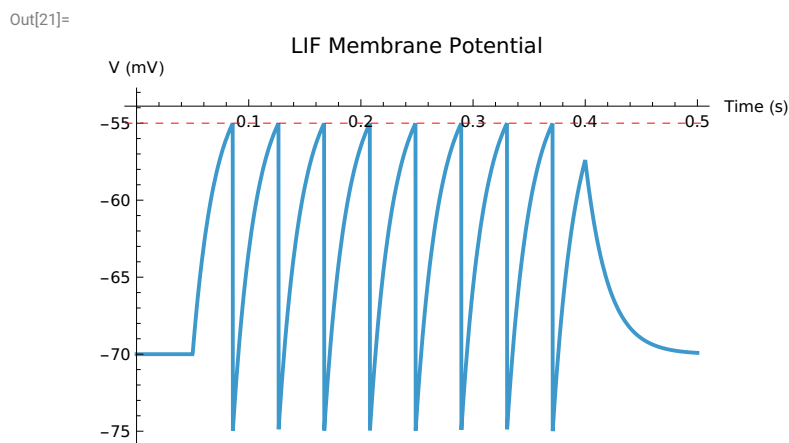
```
{0.0858352, 0.126573, 0.16731, 0.208048, 0.248786, 0.289523, 0.330261, 0.370999}
```

Explanation:

The neuron fired 8 spikes, which means the LIF model is working correctly. The membrane potential increased when the input current was turned on, reached the threshold, and then reset after each spike. The spike times occur between about 0.086 s and 0.371 s, which matches the time window when the input current is active, from 0.05 s to 0.4 s. The fairly even spacing of the spike times shows that a constant input current produces a regular firing pattern. Although the guide suggests around 10–20 spikes, 8 spikes still confirms that the neuron is integrating input and generating spikes as expected.

■ Step 5: Plot the Membrane Potential

```
In[21]:= Plot[
  Evaluate[V[t] /. sol],
  {t, 0, 0.5},
  AxesLabel → {"Time (s)", "V (mV)"},
  PlotLabel → "LIF Membrane Potential",
  Epilog → {Red, Dashed, InfiniteLine[{{0, Vthresh}, {1, Vthresh}}]},
  PlotRange → All
]
```



Explanation:

The plot shows the expected sawtooth-like membrane potential pattern of a leaky integrate-and-fire neuron. The voltage starts near the resting potential, rises toward the threshold, resets after each spike, and repeats while the input current is active. After the current turns off, the membrane poten-

tial decays back toward the resting potential. This confirms that the neuron is integrating input, firing at threshold, and resetting properly.

✓ KNOWLEDGE CHECK

1. Look at the shape of V between spikes. Is it a straight line or a curve? Why?

V between spikes is a curve, not a straight line. This happens because the LIF equation includes a leak term, $-(V[t] - V_{\text{rest}})$, which continuously pulls the membrane potential back toward the resting potential while the input current pushes it upward. Because of this balance between input and leak, the voltage rises in a curved, exponential-like way instead of linearly.

2. What happens to V after the current turns off ($t > 0.4$)? Describe what you see and explain it in terms of the LIF equation.

After the current turns off ($t > 0.4$), V decays back toward the resting potential. In the plot, the membrane potential drops smoothly toward V_{rest} and no more spikes occur. This makes sense from the LIF equation because once $\text{linput}[t]$ becomes 0, only the leak term remains, so the voltage is pulled back toward the resting potential.

Part B: Interactive Manipulate[] Dashboard

This is where Wolfram really shines. `Manipulate[]` lets you create sliders that update your simulation in real time. You can explore what happens when you change neuron parameters without rewriting any code.

Step 6: Build the Dashboard

🔧 YOUR TASK 3: Create an Interactive LIF Explorer

Wrap your `NDSolve + Plot` code inside a `Manipulate[]` with sliders for at least 3 parameters: threshold, time constant, and input current.

```
In[22]:= Manipulate[
  Module[{sol, sc = 0, st = 0, iFunc, spikeData},
```

```

(* TODO 4: Put your NDSolve code here, but replace *)
(* the fixed parameter values with the slider variables: *)
(*   Vthresh → thresh   *)
(*   tau      → tauVal  *)
(*   The constant 18.0 in Iinput → currentVal *)

(* Define a local input current function: *)
iFunc[t_] := If[0.05 ≤ t ≤ 0.4, currentVal, 0.0];

(* Then NDSolve with these local variables... *)
{sol, spikeData} = Reap[
  NDSolveValue[
    {
      tauVal * V'[t] == -(V[t] - Vrest) + R * iFunc[t],
      V[0] == Vrest,
      WhenEvent[
        V[t] ≥ thresh,
        {Sow[t], V[t] → Vreset}
      ]
    },
    V,
    {t, 0, 0.5}
  ]
];

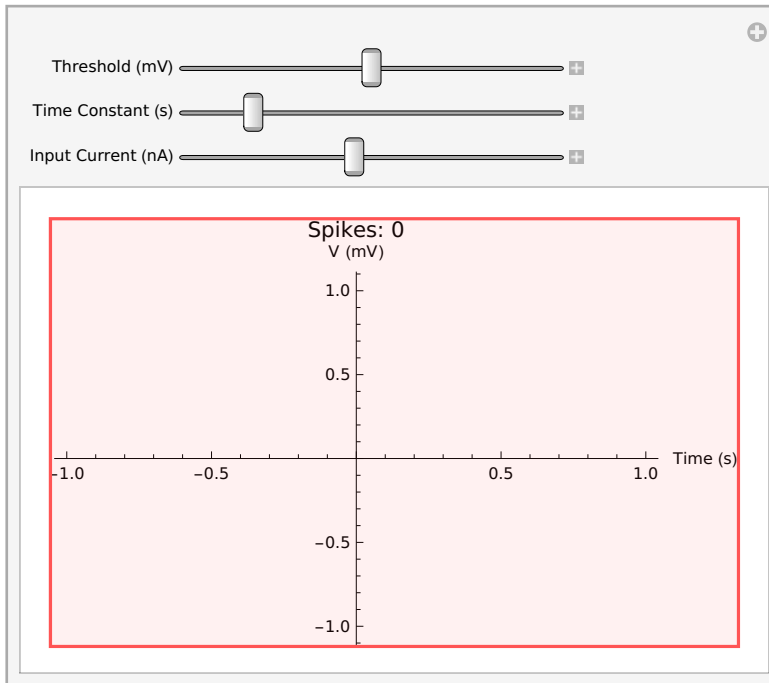
st = Flatten[spikeData];
sc = Length[st];

(* TODO 5: Plot the result *)
(* Include in the PlotLabel: how many spikes fired *)
(* Example: PlotLabel → StringForm["Spikes: ``", sc] *)
Plot[
  sol[t],
  {t, 0, 0.5},
  AxesLabel → {"Time (s)", "V (mV)"},
  PlotLabel → StringForm["Spikes: ``", sc],
  Epilog → {Red, Dashed, InfiniteLine[{{0, thresh}, {1, thresh}}]},
  PlotRange → All
],

```

```
(* Sliders *)
{{thresh, -55.0, "Threshold (mV)"}, -65, -45, 1},
{{tauVal, 0.020, "Time Constant (s)"}, 0.005, 0.100, 0.005},
{{currentVal, 18.0, "Input Current (nA)"}, 0, 40, 1}
]
```

Out[22]=



✓ KNOWLEDGE CHECK

1. Using your dashboard, find the minimum input current needed to make the neuron fire at least one spike (with default threshold and tau). This is called the rheobase. What value did you find?

The rheobase was about 16 nA. With the default settings, the neuron does not usually fire at 15 nA because the membrane potential only approaches threshold, but at about 16 nA it reaches threshold and produces at least one spike. This answer is based on the guide's default setup: $\tau = 0.020$, $V_{rest} = -70.0$, $V_{thresh} = -55.0$, $V_{reset} = -75.0$, and a current pulse from $t = 0.05$ to $t = 0.4$.

2. Set the current to 25 nA. Now slowly increase the threshold from -65 to -45 mV. At what threshold does the neuron stop firing? Why?

The neuron stops firing at about -45 mV. As I increased the threshold from -65 to -45 mV with the current fixed at 25 nA, the spikes gradually disappeared, and at about -45 mV the neuron stopped firing. This makes sense because a higher threshold means the membrane potential must rise farther to trigger a spike, and at that point the input current is no longer enough to push the voltage past

threshold during the stimulus window. The guide also notes that lowering threshold increases firing rate.

3. Set threshold to -55 and current to 20. Compare $\tau = 0.010$ vs $\tau = 0.050$. Which produces more spikes? Which produces a smoother membrane potential curve?

$\tau = 0.010$ produces more spikes, while $\tau = 0.050$ produces a smoother membrane potential curve. A smaller time constant lets the neuron respond faster, so it reaches threshold more often and fires more spikes. A larger time constant makes the voltage change more gradually, so the curve looks smoother and the firing rate is lower. This matches the guide's checkpoint that increasing τ slows down the response.

REFLECTION

One insight I gained from using the Manipulate[] dashboard was how sensitive spike behavior is to small changes in threshold and input current. Reading the booklet helped me understand the concepts, but moving the sliders made it much clearer that even a small parameter change can shift the neuron from no firing to regular spiking. The visual feedback helped me see this immediately in the membrane potential plot, especially how the spikes became more frequent when the threshold was lower or the current was higher. This made my understanding more intuitive because I could watch the neuron's behavior change in real time instead of only imagining it from the equation.

Part C: Firing Rate Curve (f-I Curve)

YOUR TASK 4: Sweep Current and Count Spikes

Write a loop that runs NDSolve for input currents from 0 to 35 nA. For each current, count how many spikes the neuron fires. Plot the result.

```
In[23]:= (* TODO 6: Generate f-I curve data *)
(* Create a list of {current, spikeCount} pairs *)

(* Hint: Use Table to loop over current values *)

fiData = Table[
  Module[{sol, spikeData, st = 0, sc = 0, iFunc},
```



```
(* define input current for this iteration *)
iFunc[t_] := If[0.05 ≤ t ≤ 0.4, Icurr, 0.0];
```

```
{sol, spikeData} = Reap[
  NDSolveValue[
    {
      tau * V'[t] == -(V[t] - Vrest) + R * iFunc[t],
      V[0] == Vrest,
      WhenEvent[
        V[t] ≥ Vthresh,
        {Sow[t], V[t] → Vreset}
      ]
    },
    V,
    {t, 0, 0.5}
  ]
];
```

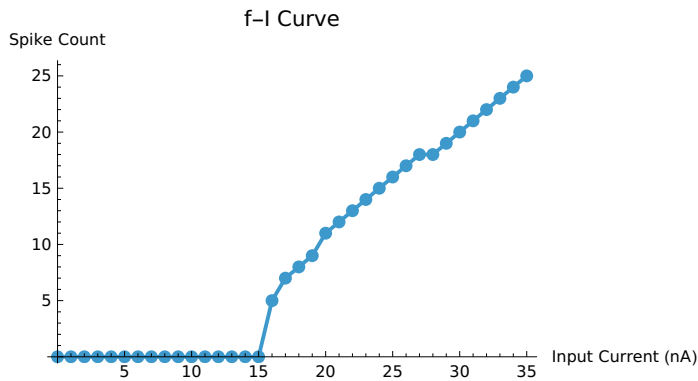
```
st = Flatten[spikeData];
sc = Length[st];
```

```
{Icurr, sc}
],
{Icurr, 0, 35, 1}
];
```

```
(* Plot the f-I curve *)
```

```
ListLinePlot[
  fiData,
  AxesLabel → {, },
  PlotLabel → ,
  PlotMarkers → Automatic
]
```

Out[24]=



✓ KNOWLEDGE CHECK

1. At what input current does your neuron first start firing? This is related to the rheobase you found with the dashboard.

From the f-I curve, the neuron first begins firing at about 15 nA. Below this value the spike count remains at zero, meaning the input current is not strong enough to push the membrane potential past the threshold. This value represents the rheobase, which is the minimum input current required for the neuron to generate spikes.

2. Is the f-I curve a straight line, or does it curve? What would a perfectly straight line mean about how the neuron encodes stimulus intensity?

The f-I curve is not perfectly straight and shows a slight curve as the input current increases. The spike count increases faster as the current becomes larger. A perfectly straight line would mean that the neuron's firing rate increases linearly with input current, meaning the neuron encodes stimulus intensity in a strictly proportional way.

3. Real neurons often show a “saturation” effect at high currents (firing rate plateaus). Does your LIF model show this? Why or why not?

In this simulation, the LIF model does not show strong saturation. The spike count continues to increase as the input current increases. This occurs because the basic LIF model does not include biological limitations such as refractory periods or ion channel dynamics that would normally limit the maximum firing rate in real neurons.

Part D: Encoding a Signal as Spikes

Now let's use your LIF neuron as an encoder. Instead of injecting constant current, we will feed it a time-varying signal (a sine wave) and observe how it converts that signal into a spike train.

⇒ YOUR TASK 5: Encode a Sine Wave as Spikes

Modify your LIF simulation to use a sine wave as input current. Plot both the input signal and the resulting spike train.

```
In[25]:= (* Define a sine wave input current *)
(* The sine wave oscillates, so we shift it to be always positive *)
Isine[t_] := 15.0 + 12.0 * Sin[2 Pi * 5 * t];
(* This gives a 5 Hz sine wave centered at 15 nA, ranging from 3 to 27 nA *)

(* TODO 7: Run NDSolve with Isine[t] as the input current *)
(* Use the same LIF equation and WhenEvent as Part A *)
(* Simulation time: 0 to 1.0 seconds *)

{solSine, spikeDataSine} = Reap[
  NDSolveValue[
    {
      tau * V'[t] == -(V[t] - Vrest) + R * Isine[t],
      V[0] == Vrest,
      WhenEvent[
        V[t] ≥ Vthresh,
        {Sow[t], V[t] → Vreset}
      ]
    },
    V,
    {t, 0, 1.0}
  ]
];

spikeTimesSine = Flatten[spikeDataSine];
spikeCountSine = Length[spikeTimesSine];

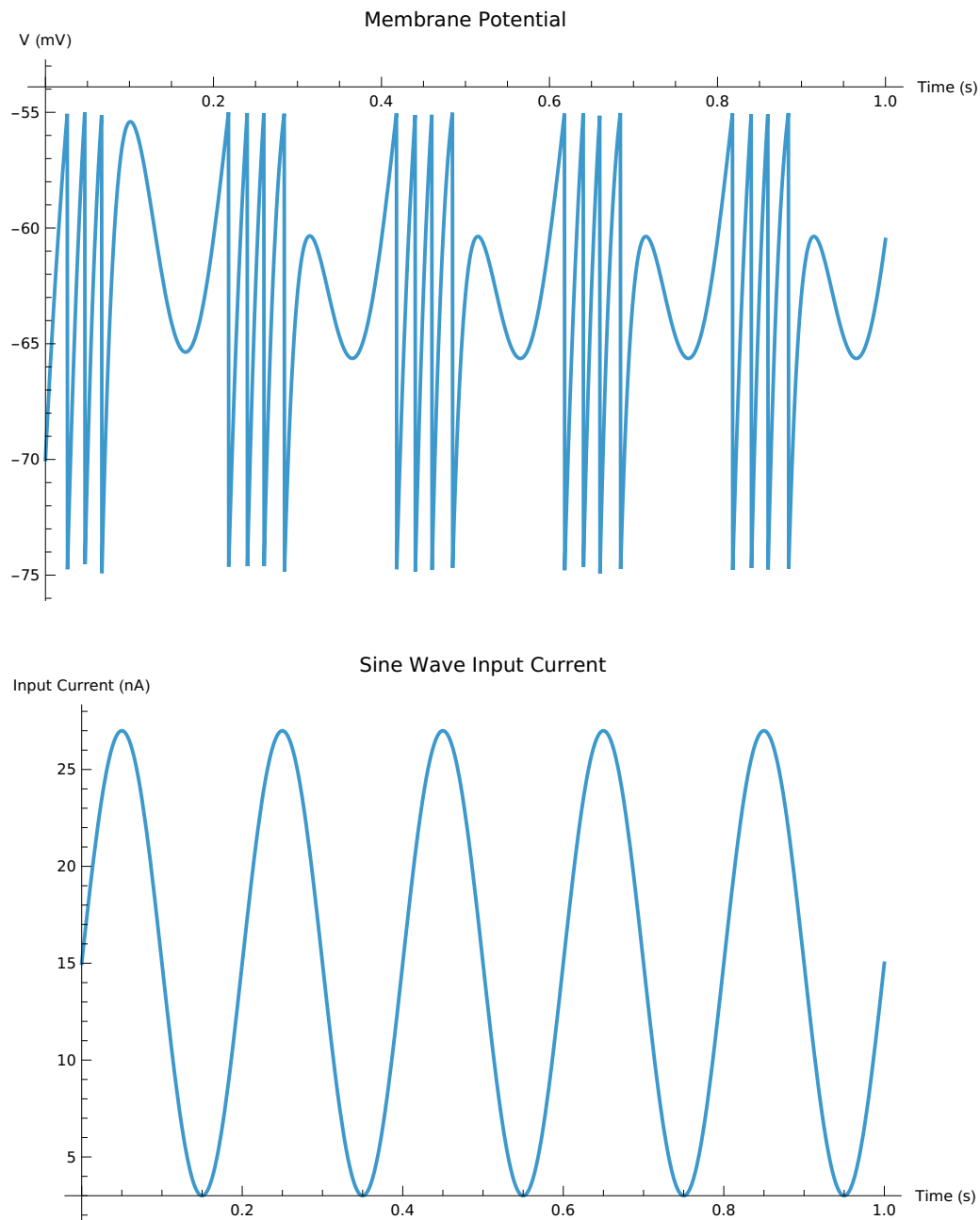
(* TODO 8: Create a two-panel plot *)
(* Top: membrane potential V[t] *)
(* Bottom: the input current Isine[t] *)
(* Hint: Use GraphicsColumn[{plot1, plot2}] or Grid *)
```

```
plot1 = Plot[
  solSine[t],
  {t, 0, 1.0},
  AxesLabel → {, },
  PlotLabel → ,
  PlotRange → All
];

plot2 = Plot[
  Isine[t],
  {t, 0, 1.0},
  AxesLabel → {"Time (s)", "Input Current (nA)"},
  PlotLabel → "Sine Wave Input Current",
  PlotRange → All
];

GraphicsColumn[{plot1, plot2}]
```

Out[31]=



REFLECTION

Look at your spike train alongside the sine wave input. The neuron is acting as an analog-to-digital converter: it takes a continuous signal and converts it into discrete spikes. How well does the spike pattern represent the original signal? What information is preserved? What is lost?

The spike pattern represents the original sine wave fairly well because the neuron fires more spikes

when the input current is high and fewer or no spikes when the input current is low. This means the spike density follows the overall shape of the sine wave, so the neuron captures the changing strength of the signal over time. The main information preserved is the timing of the high and low parts of the signal and the relative intensity of the input through firing rate. What is lost is the exact smooth shape and exact amplitude of the original sine wave, since the continuous signal is converted into discrete spikes rather than a continuous output.

Part E: Experiments

Run at least 3 of the experiments below. For each, use your Manipulate[] dashboard or modify your code, observe what happens, and document your findings in a text cell.

■ Experiment 1: Threshold Sweep

```
In[32]:= (* Experiment 1: Threshold Sweep *)
(* Change threshold from -65 to -45 in steps of 5 *)
(* Record spike count for each threshold and create a table/plot *)

thresholdData = Table[
  Module[{sol, spikeData, spikeTimes, spikeCount, threshVal},

    threshVal = th;

    {sol, spikeData} = Reap[
      NDSolveValue[
        {
          tau * V'[t] == -(V[t] - Vrest) + R * Iinput[t],
          V[0] == Vrest,
          WhenEvent[
            V[t] >= threshVal,
            {Sow[t], V[t] -> Vreset}
          ]
        },
      ],
      V,
```

```

    {t, 0, 0.5}
  ]
];

spikeTimes = Flatten[spikeData];
spikeCount = Length[spikeTimes];

{threshVal, spikeCount}
],
{th, -65, -45, 5}
];

(* Display the results as a table *)
Grid[
  Prepend[thresholdData, {, }],
  Frame → All
]

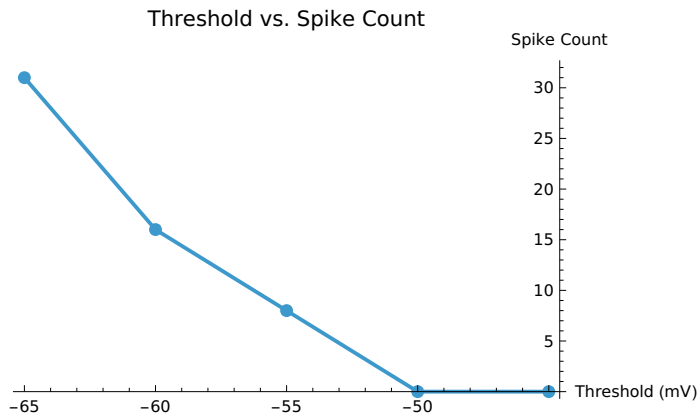
(* Plot threshold vs. spike count *)
ListLinePlot[
  thresholdData,
  AxesLabel → {, },
  PlotLabel → ,
  PlotMarkers → Automatic
]

```

Out[33]=

Threshold (mV)	Spike Count
-65	31
-60	16
-55	8
-50	0
-45	0

Out[34]=



Experiment 1: Threshold Sweep

1. What parameter did you change, and to what values?

I changed the spike threshold from -65 mV to -45 mV in steps of 5 mV. The values I tested were -65, -60, -55, -50, and -45 mV.

2. What did you expect to happen?

I expected that lower thresholds would produce more spikes because the membrane potential would not need to rise as far to trigger a spike. I also expected that higher thresholds would reduce firing or stop it completely.

3. What actually happened?

The results matched my expectation. At -65 mV, the neuron fired 31 spikes. At -60 mV, it fired 16 spikes. At -55 mV, it fired 8 spikes. At -50 mV and -45 mV, it fired 0 spikes. The plot shows a clear downward trend, meaning spike count decreased as threshold increased.

4. Why does this make sense given the LIF equation?

This makes sense because in the LIF model, a spike occurs when the membrane potential reaches the threshold. A lower threshold is easier for the neuron to reach, so the neuron spikes more often. A higher threshold requires more voltage buildup, so firing becomes less frequent. At -50 mV and -45 mV, the input current was not strong enough to raise the membrane potential to threshold, so no spikes occurred.

■ Experiment 3: Noisy Input

```

In[35]:= (* Experiment 3: Noisy Input *)
(* Replace the constant input current with a noisy current *)
(* Run the simulation several times and compare spike patterns *)

Inoisy[t_] := 18.0 + RandomReal[{-8, 8}];

{solNoisy, spikeDataNoisy} = Reap[
  NDSolveValue[
    {
      tau*V'[t] == -(V[t] - Vrest) + R*Inoisy[t],
      V[0] == Vrest,
      WhenEvent[
        V[t] ≥ Vthresh,
        {Sow[t], V[t] → Vreset}
      ]
    },
    V,
    {t, 0, 0.5}
  ]
];

spikeTimesNoisy = Flatten[spikeDataNoisy];
spikeCountNoisy = Length[spikeTimesNoisy];

Print["Neuron fired ", spikeCountNoisy, " spikes"];
Print["Spike times: ", spikeTimesNoisy];

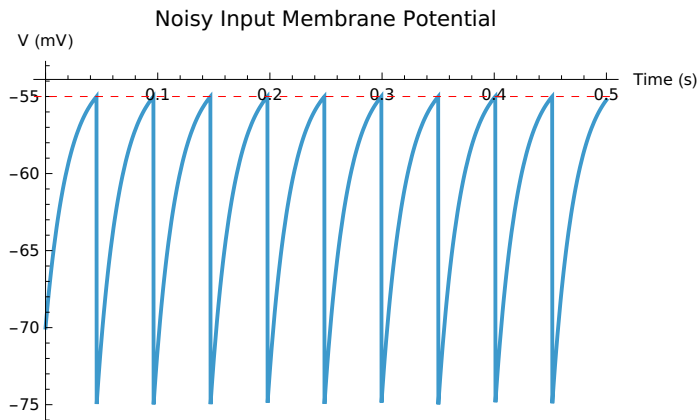
Plot[
  solNoisy[t],
  {t, 0, 0.5},
  AxesLabel → {"Time (s)", "V (mV)"},
  PlotLabel → "Noisy Input Membrane Potential",
  Epilog → {Red, Dashed, InfiniteLine[{{0, Vthresh}, {1, Vthresh}}]},
  PlotRange → All
]

```

Neuron fired 9 spikes

Spike times: {0.0455375, 0.0963089, 0.14708, 0.197852, 0.248623, 0.299395, 0.350166, 0.400937, 0.451709}

Out[41]=



Experiment 3: Noisy Input

1. What parameter did you change, and to what values?

I changed the input current from a constant current to a noisy current using $\text{Inoisy}[t_]:=18.0 + \text{RandomReal}[\{-8, 8\}]$. This means the current fluctuated randomly around 18 nA.

2. What did you expect to happen?

I expected the spike pattern to become less regular than before because the input current was changing randomly instead of staying constant. I also expected the number of spikes and their timing to vary from run to run.

3. What actually happened?

In this run, the neuron fired 9 spikes. The spike times were {0.0455375, 0.0963089, 0.14708, 0.197852, 0.248623, 0.299395, 0.350166, 0.400937, 0.451709}. The spikes were not perfectly regular, which shows that the noisy current changed how quickly the membrane potential reached threshold. When the simulation is run again, the spike pattern is not expected to be exactly the same each time.

4. Why does this make sense given the LIF equation?

This makes sense because the LIF equation depends directly on the input current. When the current includes random noise, the membrane potential rises at different rates, so the neuron reaches threshold at different times. Because `RandomReal` generates new random values, the spike pattern can change from one run to another. The experiment directions also ask you to observe whether the spike pattern is the same each time and explain why.

■ Experiment 6: Frequency Encoding

Change the sine wave frequency from 5 Hz to 2 Hz, then 10 Hz, then 20 Hz. At what frequency can the neuron no longer follow the oscillations? What does this tell you about the neuron's temporal resolution?

```
In[42]:= (* Experiment 6: Frequency Encoding *)
(* Change the sine wave frequency from 5 Hz to 2 Hz, then 10 Hz, then 20 Hz *)
(* Observe when the neuron can no longer follow the oscillations *)

freqResults = Table[
  Module[{Ifreq, sol, spikeData, spikeTimes, spikeCount, plotV, plotI},

    (* define sine wave input current for this frequency *)
    Ifreq[t_] := 15.0 + 12.0*Sin[2 Pi*f*t];

    (* run LIF simulation with this sine wave input *)
    {sol, spikeData} = Reap[
      NDSolveValue[
        {
          tau*V'[t] == -(V[t] - Vrest) + R*Ifreq[t],
          V[0] == Vrest,
          WhenEvent[
            V[t] ≥ Vthresh,
            {Sow[t], V[t] → Vreset}
          ]
        },
        V,
        {t, 0, 1.0}
      ]
    ];

    spikeTimes = Flatten[spikeData];
    spikeCount = Length[spikeTimes];

    (* create membrane potential plot *)
    plotV = Plot[
      sol[t],
      {t, 0, 1.0},
      AxesLabel → {, },
      PlotLabel → StringForm[ f],
      PlotRange → All
    ];
```

```

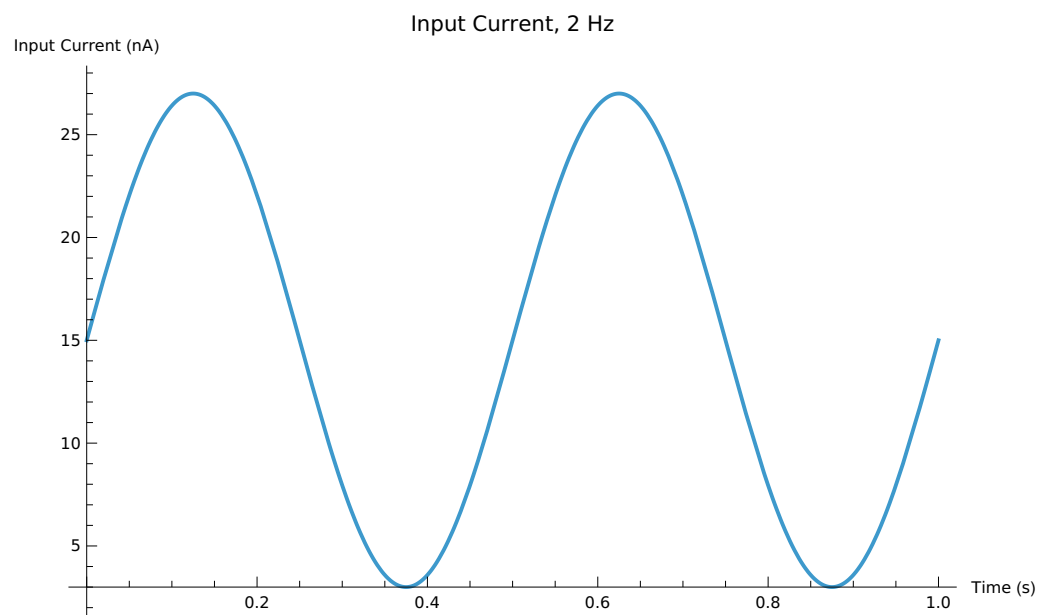
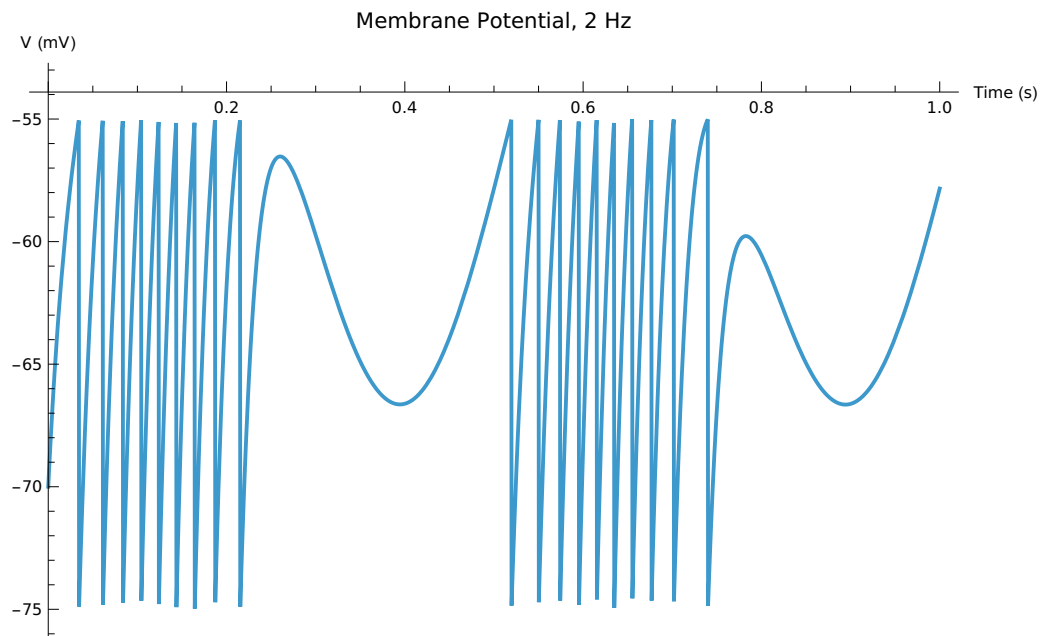
(* create input current plot *)
plotI = Plot[
  Ifreq[t],
  {t, 0, 1.0},
  AxesLabel → {, },
  PlotLabel → StringForm[, f],
  PlotRange → All
];

(* show plots for this frequency *)
Print[, f, ];
Print[, spikeCount];
Print[GraphicsColumn[{plotV, plotI}]];

(* return summary data *)
{f, spikeCount}
],
{f, {2, 5, 10, 20}}
];

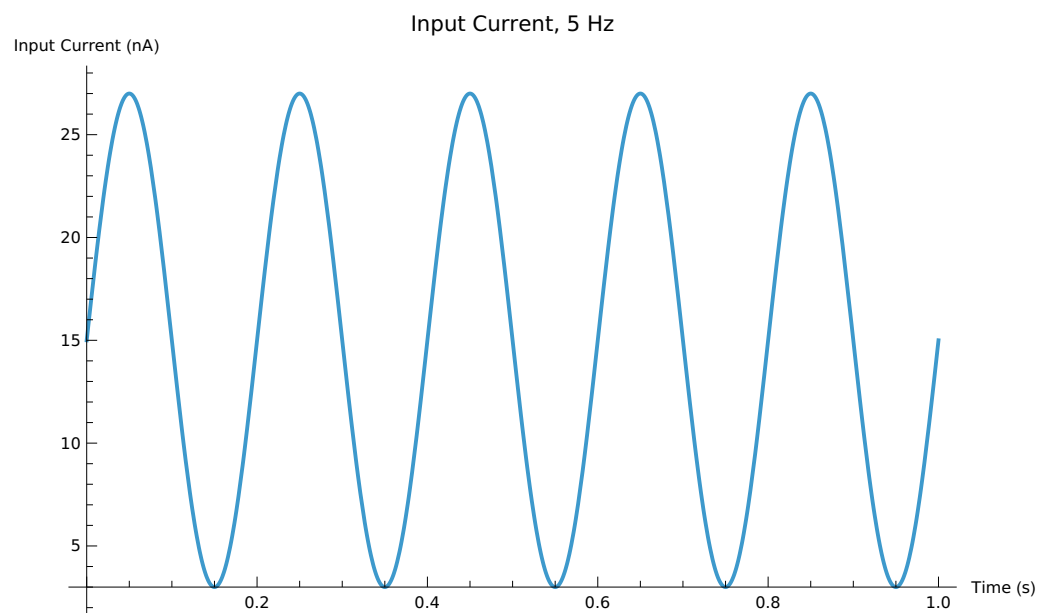
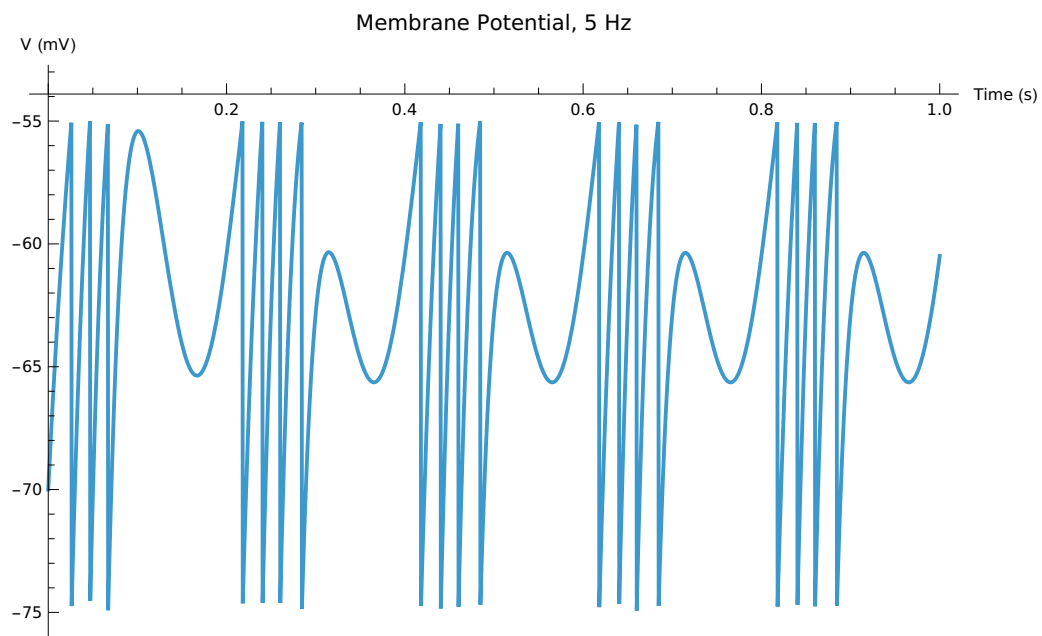
(* summary table *)
Grid[
  Prepend[freqResults, {, }],
  Frame → All
]
Frequency = 2 Hz
Spike count = 19

```



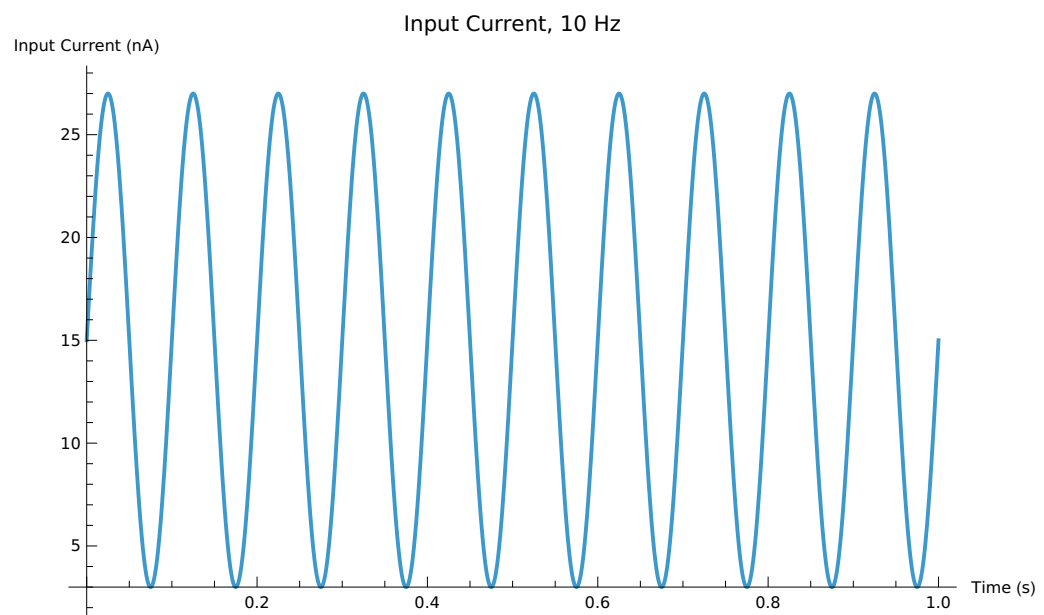
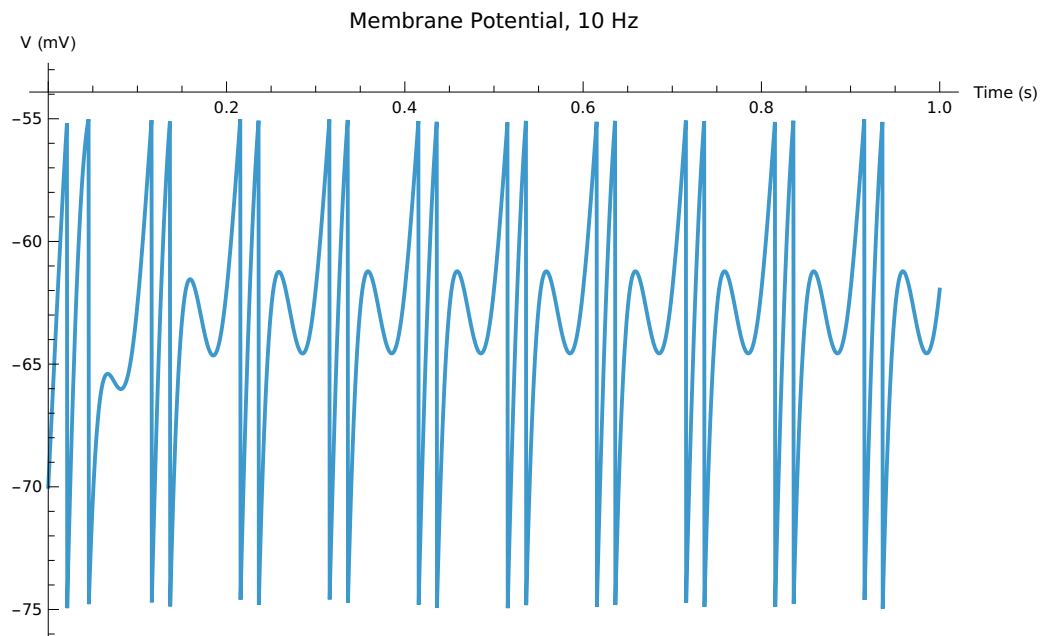
Frequency = 5 Hz

Spike count = 19



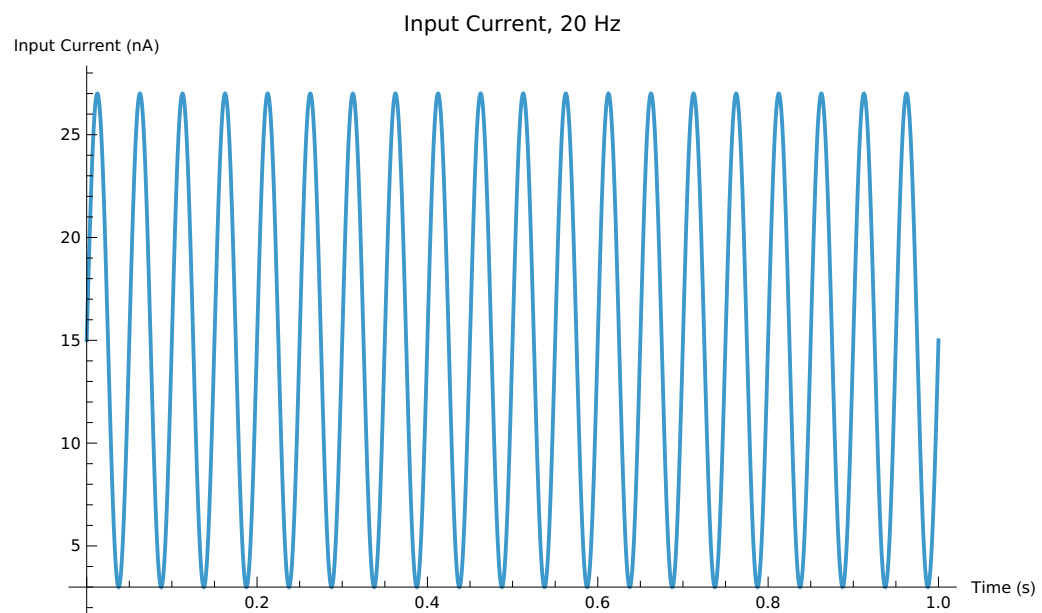
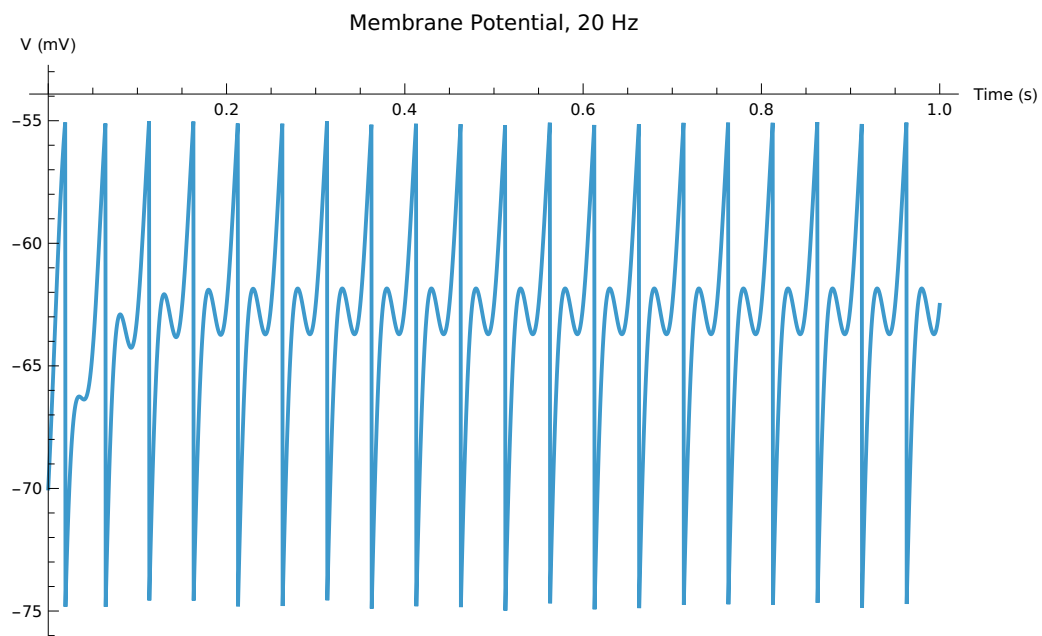
Frequency = 10 Hz

Spike count = 20



Frequency = 20 Hz

Spike count = 20



Out[43]=

Frequency (Hz)	Spike Count
2	19
5	19
10	20
20	20

Experiment 6: Frequency Encoding

1. What parameter did you change, and to what values?

I changed the sine-wave input frequency and tested 2 Hz, 5 Hz, 10 Hz, and 20 Hz while keeping the other neuron parameters the same.

2. What did you expect to happen?

I expected the neuron to follow the lower frequencies more easily because the input changes more slowly. I expected it to have more difficulty following the higher frequencies because the membrane potential needs time to rise, spike, and reset.

3. What actually happened?

At 2 Hz and 5 Hz, the neuron followed the oscillations well. The spikes were grouped around the peaks of the sine wave, and there were clearer low-spike periods near the troughs. At 10 Hz, the neuron still responded, but the spike pattern became less clearly tied to each individual oscillation. At 20 Hz, the neuron no longer followed the oscillations as clearly. The spikes became more regular and continuous, instead of being strongly grouped by each sine-wave peak. The spike counts were also very similar across frequencies, with about 19 spikes at 2 Hz and 5 Hz and 20 spikes at 10 Hz and 20 Hz.

4. Why does this make sense given the LIF equation?

This makes sense because the LIF neuron has a membrane time constant and a reset after each spike. The membrane potential cannot change instantly, so when the input oscillates too quickly, the neuron cannot track each cycle precisely. Instead of matching every oscillation, it responds more with an overall firing rate. This shows that the neuron has a limited temporal resolution.

At what frequency can the neuron no longer follow the oscillations? What does this tell you about the neuron's temporal resolution?

Based on the plots, the neuron could no longer follow the oscillations clearly at about 20 Hz. This tells me that the neuron has limited temporal resolution. It can track slower changes in the input signal, but when the frequency becomes too high, it cannot represent each oscillation accurately.

Part F: Final Reflection

1. Using your Manipulate[] dashboard, describe what happens as you gradually increase input current from below threshold to well above it. At what point does the neuron start firing? What does this threshold behavior tell you about how biological neurons make “decisions”?

As I gradually increased the input current, the membrane potential rose higher and reached the threshold more easily. When the current was below threshold, the neuron did not fire and the voltage only moved upward without producing spikes. Once the input current reached about 15–16 nA, the neuron started firing. As the current increased further above threshold, the neuron fired more frequently. This threshold behavior shows that biological neurons make “decisions” in an all-or-none way. They do not fire for every small input. Instead, they integrate incoming signals and only generate a spike when the membrane potential reaches a critical threshold.

2. Examine your f–I curve. Is the relationship linear? What biological significance does the shape of this curve have for how the brain encodes stimulus intensity?

My f–I curve was not perfectly linear. It stayed at zero for low input currents, then began rising after the rheobase, and increased more steadily in the middle range. This means the neuron does not respond equally to all input strengths. Biologically, this is important because it shows that the brain can encode stimulus intensity using firing rate. Weak inputs may produce no spikes, while stronger inputs produce more spikes. The shape of the f–I curve shows how neurons transform analog input strength into spike-based output.

3. How does changing the membrane time constant affect firing? In biological terms, what types of neurons might have short vs. long time constants, and what would each be good at?

Changing the membrane time constant changed how quickly the neuron responded to input. A smaller time constant, such as $\tau = 0.010$, produced more spikes and a faster response. A larger time constant, such as $\tau = 0.050$, produced fewer spikes and a smoother membrane potential curve. In biological terms, neurons with short time constants may be better for fast responses and detecting rapid changes, such as sensory neurons that need to respond quickly. Neurons with longer time constants may be better at integrating information over time, which could help with slower or more sustained processing.

4. Compare the LIF model to what you learned about real neurons in the booklet. What does the model capture well? What important features does it miss?

The LIF model captures several important ideas about real neurons. It shows membrane potential, resting potential, threshold behavior, leakage, spike generation, and reset after firing. These are useful for understanding the basic logic of how neurons work. However, the model also leaves out many important biological details. It does not include dendrite shape, multiple ion channels, neurotransmitter effects, adaptation, or more realistic spike shapes. Real neurons are much more complex and are influenced by many biological processes that the simple LIF model does not represent.

5. Wolfram’s Manipulate[] gave you real-time interactive exploration. How did this change your

understanding compared to just reading about neurons? What was the most surprising thing you discovered by moving the sliders?

Using Manipulate[] changed my understanding because it made neuron behavior much more visual and intuitive. Reading about the equations helped me understand the theory, but moving the sliders let me see the effect of each parameter immediately. The most surprising thing I discovered was how sensitive the neuron was to small changes in threshold and input current. A small slider adjustment could change the neuron from no spiking at all to regular firing. That real-time feedback helped me understand neuron behavior much better than reading static notes alone.

End of Lab